

Using Docker , Kubernetes and Kafka on Lab1 (Part 1 & Part 2)

Git Repo Link : https://github.com/Charishma-Cherry/DS_Lab2_Uber_Eats.git

Overview

This report describes the design, deployment, and operation of a system built using a Django backend, a React frontend, Kafka for messaging, and Kubernetes for orchestration. The system efficiently manages communication between its components, with Docker handling containerization and Kubernetes ensuring scalability and fault tolerance. Kafka facilitates real-time data sharing between microservices.

Backend Deployment

Architecture

- The Django backend is organized into three main apps:
 - **customers:** Handles customer-related features.
 - **restaurants:** Manages restaurant operations.
 - **orders:** Processes orders.
- These apps are deployed as **separate containers** within a single Kubernetes pod called the **backend pod**. This structure ensures that all backend functionalities are grouped together while maintaining modularity.

Kafka Integration

1. Consumers:

- **Customers App:**
 - Contains a Kafka consumer script called `customers_kafka_consumer.py`.
 - Listens to the **order_updates** topic, receiving updates like order status changes (e.g., order delivered, order canceled).
- **Restaurants App:**
 - Contains a Kafka consumer script called `restaurants_kafka_consumer.py`.
 - Listens to the **orders** topic, receive messages about new orders placed by customers.

2. Producer:

- The orders app includes a Kafka producer script, `producer.py`, which sends messages to Kafka topics:

- **orders:** For notifying restaurants of new orders.
- **order_updates:** For updating customers on their order status.

3. Message Flow:

- The orders app generates messages and pushes them to Kafka topics.
- These messages are consumed by the customers and restaurants apps, ensuring real-time updates between users, restaurants, and the system.

Kafka Setup

- Kafka is configured to operate at **kafka-service:9092**, with Zookeeper managing the Kafka cluster.
 - **Topics Used:**
 - **orders:** Sends new order details to restaurants.
 - **order_updates:** Sends order status updates to customers.
-

Frontend Deployment

- The React frontend is deployed in its own Kubernetes pod as a Dockerized container.
 - It communicates with the backend through REST APIs to manage user interactions, such as placing orders or viewing updates.
-

Kubernetes Orchestration

Cluster Details

- Kubernetes is deployed **locally on Mac** using **Minikube**.
- Nginx is configured within the cluster to route traffic efficiently between services.

Pods and Containers

1. Backend Pod:

- This pod contains multiple containers for each Django app (customers, restaurants, orders) and Nginx.
- Nginx acts as a reverse proxy to manage traffic between the backend containers.

2. Frontend Pod:

- Runs the React frontend in a separate container.

3. Kafka Service:

- Kafka and Zookeeper are deployed in their own services within the cluster to ensure independent message brokering and scalability.

4. MySQL:

- MySql pod runs MySql server and exposes “mysql-service” at port 3306 in the cluster.
- All services in the backend pod connect to the same mysql database.

Service Configuration

- Kubernetes services manage communication between pods and provide load balancing.
 - Kafka handles asynchronous message exchanges between services (restaurants , orders and customers).
-

Kafka Message Pipeline

Producer (Orders App)

- The Kafka producer in the orders app serializes data into JSON format and sends it to Kafka topics:
 - **orders** for new order messages.
 - **order_updates** for order status updates.

Consumers

- **Customers Kafka Consumer (customers_kafka_consumer.py):**
 - Subscribes to the **order_updates** topic to get order updates for customers.
- **Restaurants Kafka Consumer (restaurants_kafka_consumer.py):**
 - Subscribes to the **orders** topic to receive details about new orders.

Message Handling

- Kafka topics are well-organized to keep messages separated based on their purpose:
 - **orders**: New orders for restaurants.
 - **order_updates**: Updates about existing orders for customers.
-

Dockerized Components

Backend

- Each Django app (customers, restaurants, orders) is containerized with its dependencies using Docker.

- Docker Compose is used for local development and testing, simplifying the setup process.

Frontend

- The React app is containerized using a **multi-stage Docker build**, which optimizes the image size for production deployment.

Kafka and Zookeeper

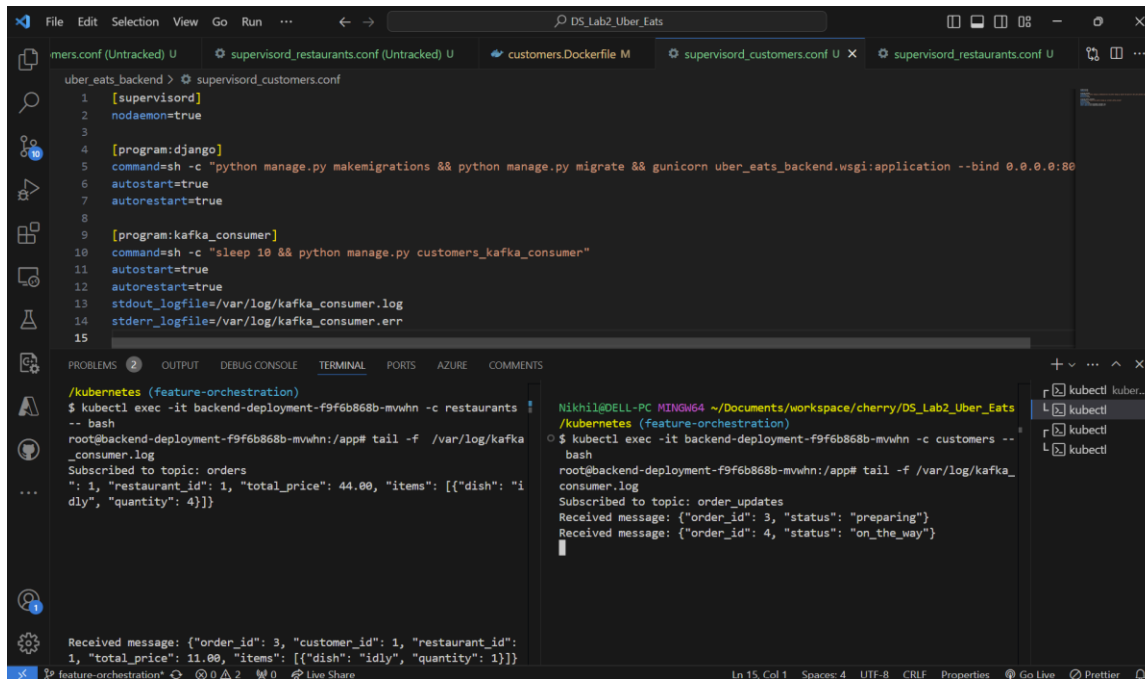
- Kafka and Zookeeper are also containerized using Docker, making them easy to deploy within the Kubernetes cluster.

System Summary

- **Backend:** Handles core logic and is modular, with each app in a separate container. Kafka enables real-time messaging.
- **Frontend:** Deployed in its own pod, ensuring user interactions are seamless.
- **Kafka:** Acts as the backbone for communication, ensuring real-time updates between services.
- **Kubernetes:** Provides a robust orchestration framework for managing pods, scaling services, and ensuring efficient communication.
- **Docker:** Simplifies deployment by packaging all components with their dependencies.

This system is designed to handle real-time communication, is scalable for future needs, and ensures efficient resource usage through Kubernetes orchestration and Docker containerization.

Screenshots :



The screenshot shows a VS Code editor with a file explorer on the left and a terminal at the bottom. The file explorer shows a project named 'DS_Lab2_Uber_Eats' with files: 'mers.conf (Untracked) U', 'supervisord_restaurants.conf (Untracked) U', 'customers.Dockerfile M', 'supervisord_customers.conf U', and 'supervisord_restaurants.conf U'. The 'customers.Dockerfile' is open in the editor, showing a Dockerfile for a Django application with a Kafka consumer. The terminal at the bottom shows the output of a 'kubectl exec' command, displaying the logs of the 'customers' pod. The logs show the application subscribing to the 'orders' topic and receiving messages. The messages are JSON objects containing order details like 'order_id', 'customer_id', 'restaurant_id', 'total_price', and 'items'.

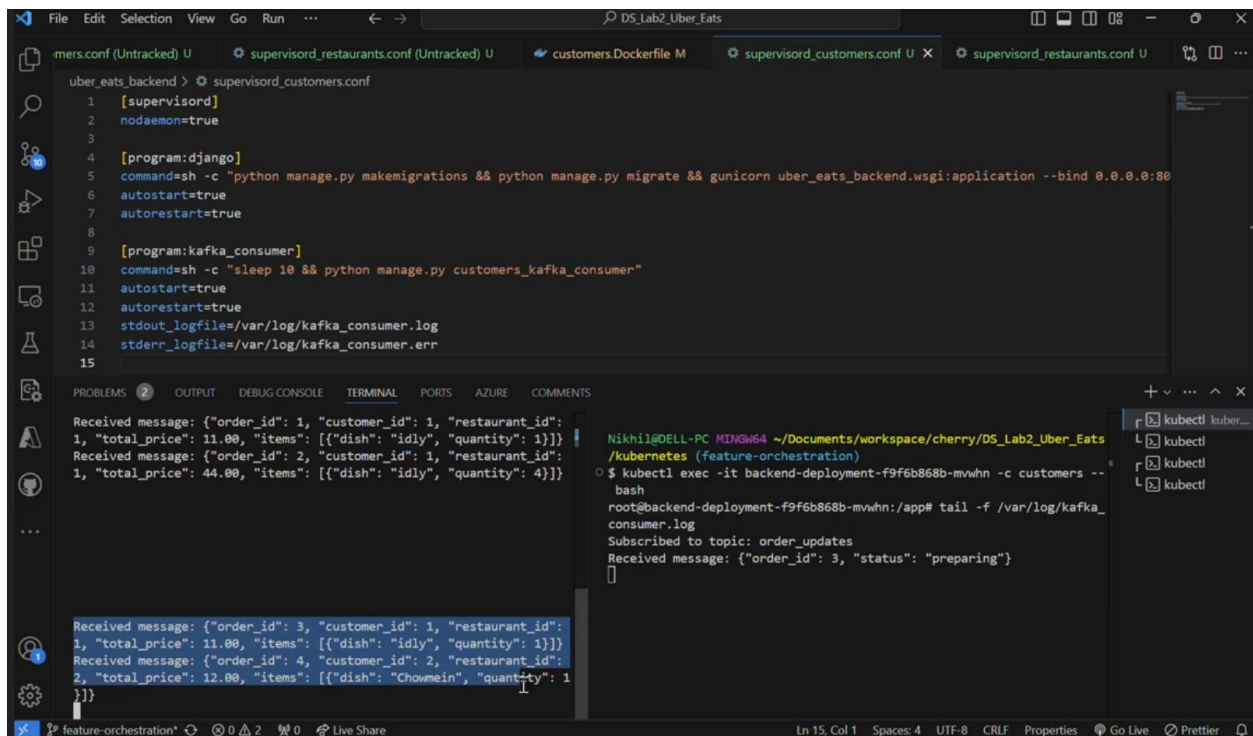
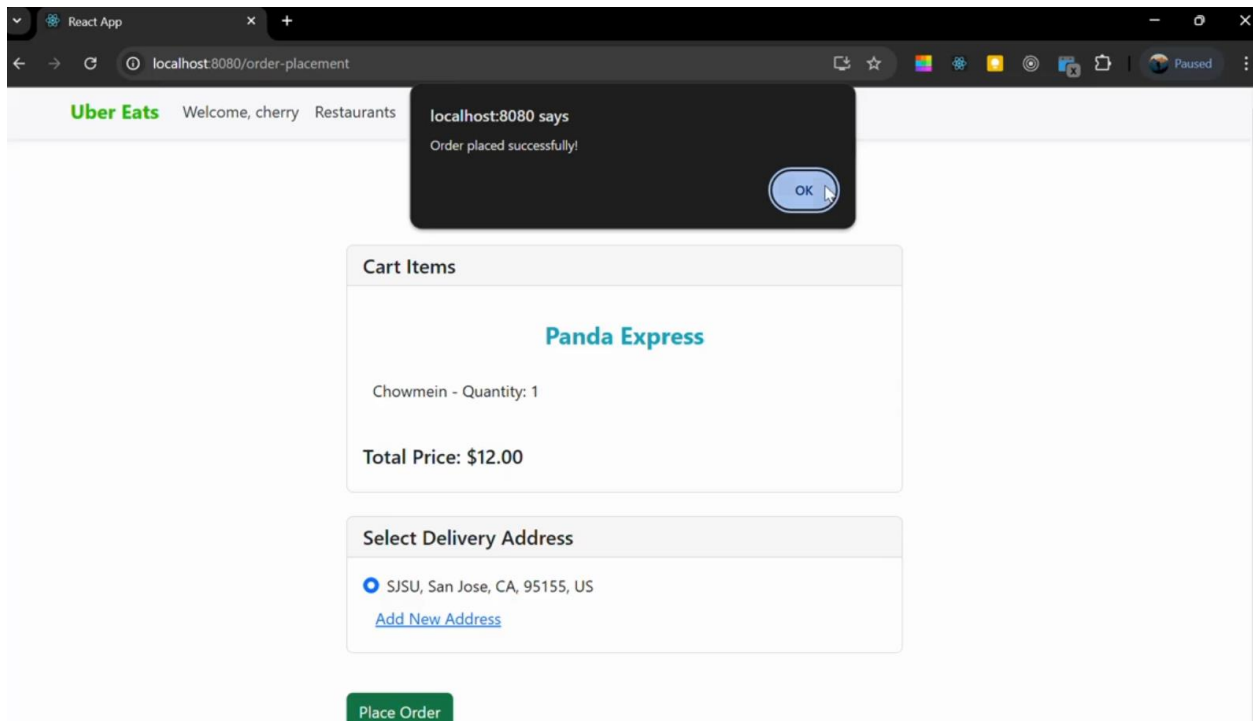
```
1 [supervisord]
2 nodaemon=true
3
4 [program:django]
5 command=sh -c "python manage.py makemigrations && python manage.py migrate && gunicorn uber_eats_backend.wsgi:application --bind 0.0.0.0:80
6 autostart=true
7 autorestart=true
8
9 [program:kafka_consumer]
10 command=sh -c "sleep 10 && python manage.py customers_kafka_consumer"
11 autostart=true
12 autorestart=true
13 stdout_logfile=/var/log/kafka_consumer.log
14 stderr_logfile=/var/log/kafka_consumer.err
15
```

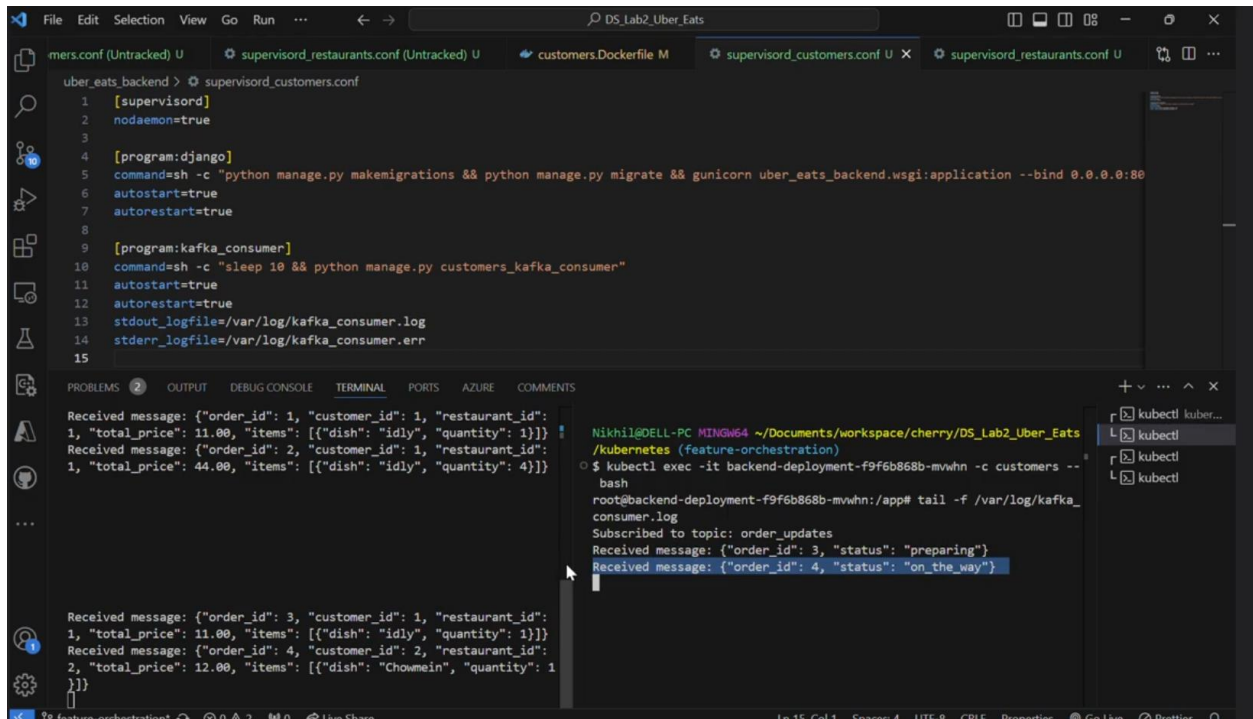
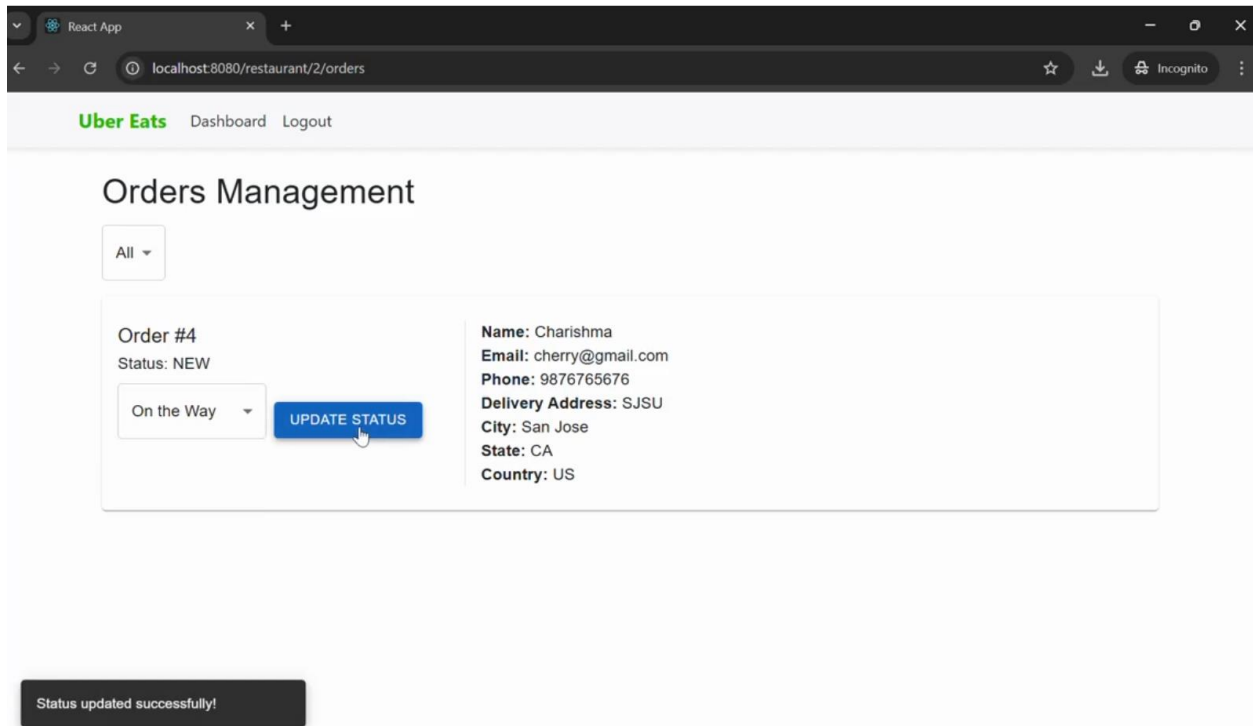
```
/kubernetes (feature-orchestration)
$ kubectl exec -it backend-deployment-f9f6b868b-mvwhn -c restaurants -- bash
root@backend-deployment-f9f6b868b-mvwhn:/app# tail -f /var/log/kafka_consumer.log
Subscribed to topic: orders
{"order_id": 1, "restaurant_id": 1, "total_price": 44.00, "items": [{"dish": "idly", "quantity": 4}]}

Received message: {"order_id": 3, "customer_id": 1, "restaurant_id": 1, "total_price": 11.00, "items": [{"dish": "idly", "quantity": 1}]}

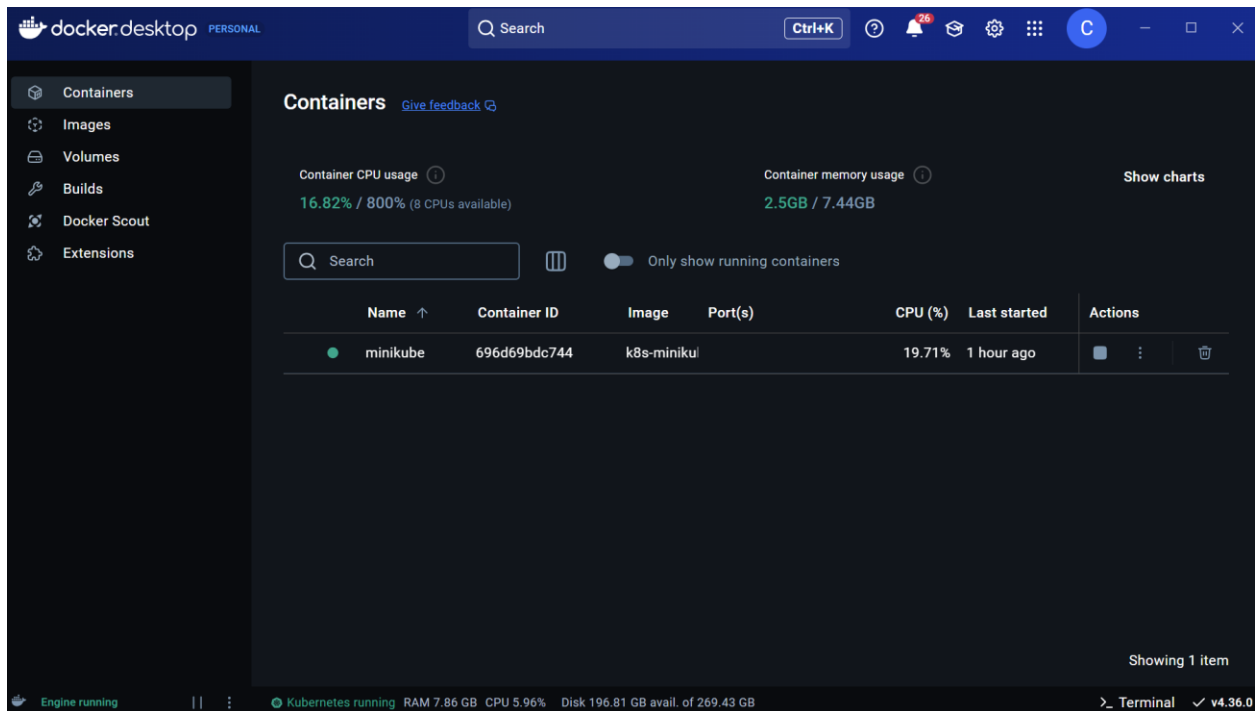
Nikhil@DELL-PC MINGW64 ~/Documents/workspace/cherry/DS_Lab2_Uber_Eats
/kubernetes (feature-orchestration)
$ kubectl exec -it backend-deployment-f9f6b868b-mvwhn -c customers -- bash
root@backend-deployment-f9f6b868b-mvwhn:/app# tail -f /var/log/kafka_consumer.log
Subscribed to topic: order_updates
Received message: {"order_id": 3, "status": "preparing"}
Received message: {"order_id": 4, "status": "on_the_way"}
```

Kafka Consumer logs of **orders** topic (when order is created on user , notifying the message on restaurant side) and **order_updates** (when the order status is changed on restaurant side , notifying the message on user side)

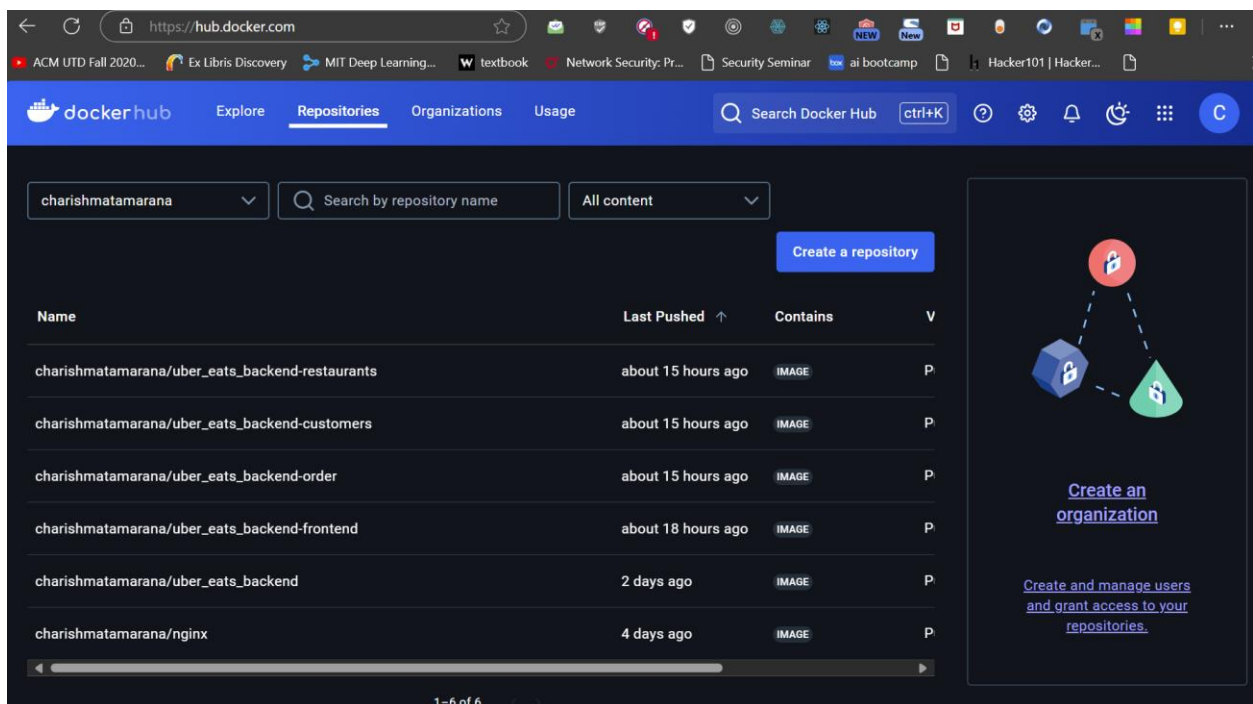




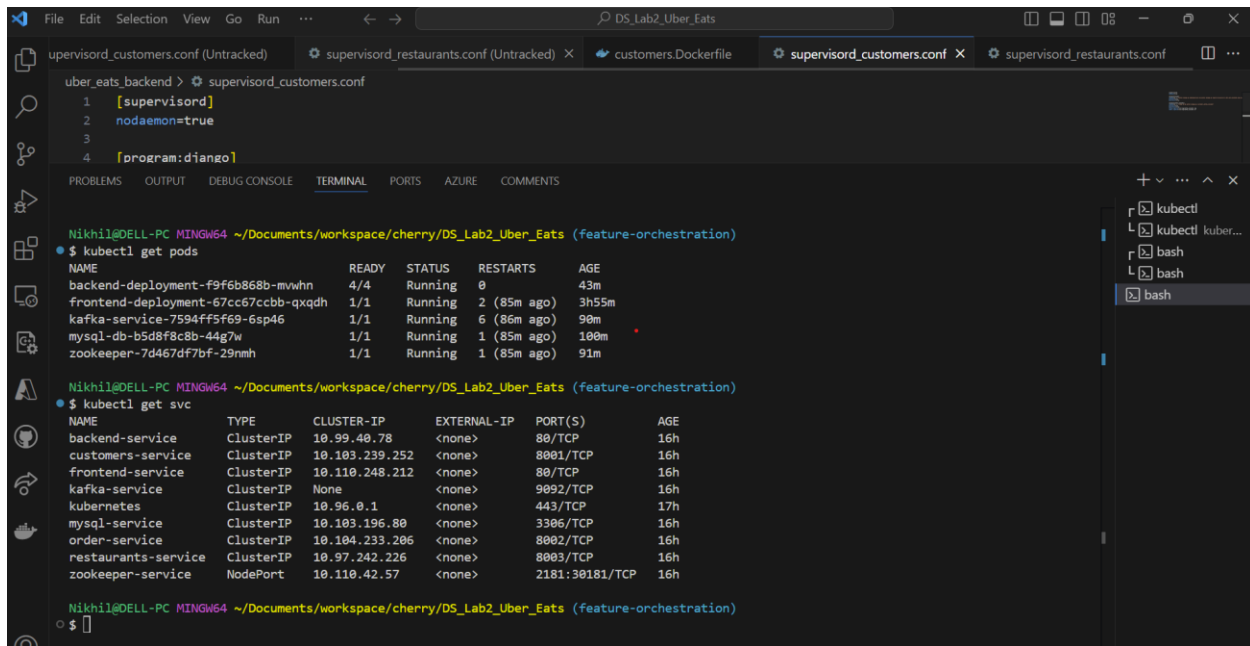
Docker Desktop containing the Kubernetes container minikube.



On docker hub , the docker images created for backend restaurants , customers , order , frontend and nginx.



The pods and services created which are in running status



The screenshot shows a Visual Studio Code editor with a terminal window open. The terminal displays the output of two Kubernetes commands: `kubectl get pods` and `kubectl get svc`. The `kubectl get pods` command shows five pods in a 'Running' state: `backend-deployment-f9f6b86b-mvwhn`, `frontend-deployment-67cc67ccbb-qxqdh`, `kafka-service-7594ff5f69-6sp46`, `mysql-db-b5d8f8c8b-44g7w`, and `zookeeper-7d467df7bf-29nmh`. The `kubectl get svc` command shows seven services: `backend-service`, `customers-service`, `frontend-service`, `kafka-service`, `kubernetes`, `mysql-service`, `order-service`, `restaurants-service`, and `zookeeper-service`. The `kubernetes` service is of type `ClusterIP` and has a port of `443/TCP`. The `zookeeper-service` is of type `NodePort` and has a port of `2181:30181/TCP`. The terminal also shows the command `kubectl get pods` being executed again, and the `kubectl get svc` command being executed again.

```
upervisord_customers.conf (Untracked)  supervisord_restaurants.conf (Untracked)  customers.Dockerfile  supervisord_customers.conf  supervisord_restaurants.conf

uber_eats_backend > supervisord_customers.conf
1 [supervisord]
2 nodaemon=true
3
4 [program:django]

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  AZURE  COMMENTS

Nikhil@DELL-PC MINGW64 ~/Documents/workspace/cherry/DS_Lab2_Uber_Eats (feature-orchestration)
$ kubectl get pods
NAME                                READY    STATUS    RESTARTS   AGE
backend-deployment-f9f6b86b-mvwhn    4/4      Running   0           43m
frontend-deployment-67cc67ccbb-qxqdh 1/1      Running   2 (85m ago) 3h55m
kafka-service-7594ff5f69-6sp46       1/1      Running   6 (86m ago) 90m
mysql-db-b5d8f8c8b-44g7w             1/1      Running   1 (85m ago) 100m
zookeeper-7d467df7bf-29nmh           1/1      Running   1 (85m ago) 91m

Nikhil@DELL-PC MINGW64 ~/Documents/workspace/cherry/DS_Lab2_Uber_Eats (feature-orchestration)
$ kubectl get svc
NAME                                TYPE        CLUSTER-IP    EXTERNAL-IP  PORT(S)    AGE
backend-service                     ClusterIP    10.99.40.78    <none>        80/TCP     16h
customers-service                   ClusterIP    10.103.239.252 <none>        8001/TCP   16h
frontend-service                    ClusterIP    10.110.248.212 <none>        80/TCP     16h
kafka-service                       ClusterIP    None           <none>        9092/TCP   16h
kubernetes                         ClusterIP    10.96.0.1      <none>        443/TCP    17h
mysql-service                       ClusterIP    10.103.196.80  <none>        3306/TCP   16h
order-service                       ClusterIP    10.104.233.206 <none>        8002/TCP   16h
restaurants-service                 ClusterIP    10.97.242.226  <none>        8003/TCP   16h
zookeeper-service                   NodePort     10.110.42.57   <none>        2181:30181/TCP 16h

Nikhil@DELL-PC MINGW64 ~/Documents/workspace/cherry/DS_Lab2_Uber_Eats (feature-orchestration)
$
```